PAGE & PAPER – E-BOOK SALES AND PREVIEW WEBSITE

A RESPONSIVE MULTI-PAGE E-BOOK STORE USING HTML, CSS, BOOTSTRAP AND JAVASCRIPT

CS23531 - MINI-PROJECT REPORT

Submitted by

PUVIYARASI S (241001181)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **PAGE & PAPER – E-BOOK SALES AND PREVIEW WEBSITE: A RESPONSIVE MULTI-PAGE E-BOOK STORE USING HTML, CSS, BOOTSTRAP AND JAVASCRIPT** is the bonafide work of **PUVIYARASI S (241001181)**, who carried out the work during the Mini-Project under supervision. Certified further that, based on the task given by the training organisation, the project was completed under their supervision.

SIGNATURE OF THE CLASS IN-CHARGE

SIGNATURE OF THE REVIEWER

SIGNATURE OF THE HOD

Dr. P. Neelaveni
Head of the Department
Department of Information Technology
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our chairman Mr. S. Meganathan, B.E., F.I.E., for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson Dr. Thangam Meganathan, for her enthusiastic motivation which inspired us a lot in completing this project and Vice-Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college principal, Dr. S. N. Murugesan, M.E., Ph.D., for his kind support and facilities to complete our work on time. We extend heartfelt gratitude to Dr. P. Neelaveni, Professor and Head of the Department of Information Technology, for her guidance and encouragement throughout the work.

We extend our thanks to our project guide, faculty members, parents, and friends for their direct and indirect involvement in the successful completion of this project, for their encouragement and support.

PUVIYARASI S

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
—	Abstract	5
1	Introduction	5
2	Literature Review and Background Study	6
3	System Requirements	7
4	System Design and Methodology	8
5	Technology Stack	10
6	Implementation	11
7	Results and Discussion	15
8	Challenges and Issues Faced	17
9	Testing	18
—	Appendix A — Markup Excerpts (HTML pages)	19
—	Appendix B — Design Tokens and Responsive Rules (style.css)	20
—	Appendix C — Interaction Logic (JavaScript)	20
10	Conclusion and Future Scope	21
11	References	22
12	System Usage / Use Case Description	22
13	User Manual — Running the Application	23
14	Glossary of Terms	24
—	SDG Mapping	24

Abstract

PAGE & PAPER is a responsive, multi-page online e-book bookstore built as a front-end demonstration. It lets a visitor browse a catalogue of eight e-books, search and filter them by category, read the full details of any title, open a sample reading preview, collect books in a shopping cart and send an enquiry through a contact form. The project shows how a complete and professional-looking store experience can be delivered with HTML5, CSS3, Bootstrap 5 and JavaScript alone, with no server-side code, no database and no build step. The site is organised into six linked pages — Home, Explore Books, Book Details, Book Preview, Shopping Cart and Contact — and adapts to desktop, tablet and mobile screens.

The application was developed in three stages. In the first stage, the information architecture and the visual identity were defined: six pages sharing one stylesheet, a common navigation bar and footer, and a set of colour and spacing tokens stored as CSS custom properties. In the second stage, the catalogue was implemented as a data-driven module: one JavaScript array of book objects drives the featured section, the catalogue cards, the category buttons, the live search, the details page and the preview page. Eight original SVG illustrations serve as book covers. In the third stage, the shopping cart was added. It stores books and quantities in the browser's localStorage, keeps every quantity between 1 and 10, removes books on request and recalculates line totals and the grand total after every change.

The project is deliberately limited to the browser. The cart is saved only in the visitor's own browser, the contact form validates its fields but does not transmit them, and the site has no payment gateway, authentication or order processing. These limits are stated openly in the report together with the test cases used to check the site and recommended improvements for a production version.

1. Introduction

Reading has moved steadily from paper to screens. Readers now expect to find a book, learn who wrote it, see what it costs and try a few pages before they decide to buy — all in a few clicks and on whatever device they happen to hold. A bookstore website therefore has to do more than list titles. It must make books easy to find, present each one clearly, let the visitor sample the reading experience and keep track of what has been chosen.

PAGE & PAPER – E-Book Sales and Preview Website addresses this need with a purpose-built, multi-page front end for an online e-book store. The Home page introduces the store and highlights featured books. The Explore Books page presents the whole catalogue with a search box and category filters. The Book Details page shows the author, description and price of one selected title, and the Book Preview page offers a sample of its content in a simple reading view. The Shopping Cart page collects the chosen books, lets the visitor change quantities within a limit of 1 to 10 or remove books, and calculates the totals. The Contact page gives the store's contact information and an enquiry form.

The project restricts itself to client-side web technologies. HTML5 gives each page its structure, CSS3 and a shared stylesheet give the pages one consistent look, Bootstrap 5 provides the responsive grid and ready-made components such as the navigation bar, cards, buttons and form controls, and JavaScript supplies the behaviour: shared navigation and footer, search and filtering, details and preview loading, form validation and the cart. Because everything runs in the browser, the site is light, easy to understand and easy to host on any static server. It is a front-end demonstration of an e-commerce interface; it does not process real payments or real orders. This report documents the background, requirements, design, implementation, results, testing, challenges and future scope of the system.

2. Literature Review and Background Study

2.1 Online Bookstores and E-books

Online bookstores have become a standard way to buy books, and e-books in particular remove the need for printing, storage and shipping. Studies of online retail interfaces commonly point to the same essentials: a clear catalogue, a visible search function, a product page that answers the buyer's basic questions, and a cart whose contents and totals are always easy to check. For books specifically, a sample of the text is regarded as one of the strongest aids to a buying decision because it lets the reader judge the writing style before paying.

2.2 Responsive Web Design and Bootstrap

Responsive web design, introduced by Ethan Marcotte in 2010, combines fluid grids, flexible images and CSS media queries so that one code base adapts to any screen size. Bootstrap is a widely used front-end framework that packages this approach. Bootstrap 5 uses a mobile-first, twelve-column grid with six breakpoints (extra small, small, medium, large, extra large and extra-extra large), a set of utility classes for spacing, alignment and display, and components such as the responsive navbar that collapses into a menu button on small screens. Version 5 no longer depends on jQuery, which makes it well suited to a project written in plain JavaScript.

2.3 Multi-Page Websites and Shared Components

A multi-page website splits its content over several HTML documents connected by hyperlinks. It suits a store well because each task — browsing, reading about a book, sampling it, reviewing the cart — has its own address that can be bookmarked and reached with the browser's back and forward buttons. The drawback is repetition: every page needs the same navigation bar and footer. Without a server-side template engine, this repetition can be removed by writing the shared markup once in JavaScript and inserting it into placeholder elements when each page loads, so that a change to the menu has to be made in one place only. Data can be passed between pages through the query string of the URL, for example book-details.html?id=3, which is read with the URLSearchParams interface.

2.4 Client-Side Storage and the Shopping Cart Pattern

The Web Storage API provides localStorage, a key–value store that keeps strings in the browser for a single origin and survives page reloads and browser restarts. Typical capacity is about five megabytes. Values are stored as text, so structured data such as a cart is converted with JSON.stringify and restored with JSON.parse. A cart kept in localStorage is shared by all pages of the site and persists between visits, but it exists only in one browser on one device and offers no security. It is therefore suitable for a demonstration, whereas a real store keeps the cart and the order on a server.

2.5 Scalable Vector Graphics

Scalable Vector Graphics (SVG) is a W3C standard for describing two-dimensional images with XML markup. Because shapes, gradients and text are described mathematically rather than as pixels, an SVG image stays sharp at any size, is usually small in file size and can be styled or generated with code. These properties make SVG a good choice for original book-cover illustrations that must look crisp on both small phones and large monitors, without depending on third-party stock images.

2.6 Usability, Accessibility and Form Validation

The Web Content Accessibility Guidelines (WCAG) 2.2 describe how to make content perceivable, operable, understandable and robust. Practical measures include semantic landmarks, text alternatives for images, labelled form controls, visible keyboard focus and sufficient colour contrast. Good form design validates input at the right moment, writes the problem in plain language next to the field and does not discard what the user has typed. Numeric inputs such as a quantity should be limited to a sensible range and corrected automatically when a value outside the range is entered.

2.7 Summary

The study shows that (i) an online bookstore needs a findable catalogue, a clear product page, a sample of the content and a trustworthy cart, (ii) a responsive, data-driven, standards-based front end built on a framework such as Bootstrap can deliver this without a back end for demonstration purposes, and (iii) accessibility and careful validation are part of the user experience. These conclusions shaped the design decisions described in the following chapters.

3. System Requirements

3.1 Hardware Requirements

- Any desktop, laptop, tablet or smartphone able to run a current web browser; no special hardware is needed
- Approximately 4 GB RAM and a standard display for comfortable development and testing
- Internet connectivity is needed only to load the Bootstrap 5 files from a content delivery network (see 3.3); the pages themselves can be opened from local files

3.2 Software Requirements

- A current version of Google Chrome, Microsoft Edge or Mozilla Firefox (CSS custom properties, CSS Grid, Flexbox, SVG, localStorage and URLSearchParams are used)
- A text or code editor for development, for example Visual Studio Code
- Optional: a simple local web server (such as python -m http.server); the site also opens directly from the file system, although a server gives the most reliable localStorage behaviour
- The browser's developer tools, used to inspect the DOM, the console and the stored cart data

3.3 External Resources Used

- Bootstrap 5 — CSS and JavaScript bundle linked from a CDN, providing the grid, navbar, cards, buttons, forms, badges and utilities
- Eight original SVG illustrations — one cover for each book, created for this project
- Browser localStorage — stores the shopping cart under the key pagepaper_cart
- No stock images, icon libraries, back-end services or payment services are used

3.4 Functional Requirements

- Show a Home page with an introduction to the store, featured books and clear navigation to the other pages
- Show a catalogue of eight books on the Explore Books page, with live search and category filtering that can be combined
- Open any book on a Book Details page that shows its cover, title, author, category, description and price
- Open a Book Preview page that presents a sample of the e-book in a reading view with page navigation
- Let the visitor add books to a Shopping Cart, change quantities between 1 and 10, remove books and see line totals and the grand total
- Provide a Contact page with contact information and an enquiry form with field validation
- Show the same navigation bar and footer on every page, including a live cart item count

3.5 Non-Functional Requirements

- Responsive — usable from 320 px phone screens to large desktop monitors
- Consistent — one shared stylesheet and one shared navigation and footer for all pages
- Portable — runs without installation or build tools as plain HTML, CSS, JavaScript and SVG files

- Maintainable — all book information held in one data array so that titles can be added or edited in one place
- Honest — the interface and the report do not claim real payment, login or order processing

4. System Design and Methodology

4.1 Overview of the Development Approach

The website was developed iteratively in three stages: (1) page structure, shared layout and visual identity, (2) the data-driven catalogue with search, filtering, details and preview, and (3) the shopping cart and the contact form, followed by polishing for responsiveness and accessibility. The project is organised into HTML pages, one stylesheet, a small set of JavaScript files and an assets folder of SVG covers, with the responsibilities summarised in Table 4.1.

File / folder	Responsibility
index.html	Home page: hero introduction, featured books and calls to action
explore.html	Explore Books page: search box, category filters, result counter and catalogue grid
book-details.html	Book Details page: loads one book from the query string and shows its full information
preview.html	Book Preview page: sample e-book reading view with page navigation
cart.html	Shopping Cart page: item list, quantity controls, remove buttons and totals
contact.html	Contact page: contact information panel and validated enquiry form
css/style.css	Shared stylesheet: design tokens, component styles and responsive adjustments on top of Bootstrap
js/books.js	The books array: the single source of data for all pages
js/layout.js	Shared navigation bar and footer inserted into every page
js/cart.js	Cart functions: load, save, add, change quantity, remove, totals and badge count
js/pages.js	Page-specific logic: featured list, catalogue, details, preview and contact form
assets/covers/	Eight original SVG book-cover illustrations

Table 4.1 — Files of the PAGE & PAPER front end

4.2 Information Architecture

The site consists of six pages linked by the shared navigation bar. Pages are connected by ordinary hyperlinks; the selected book is passed to the details and preview pages through the id parameter of the URL.

Page	Address	Purpose and key content
Home	index.html	Store name and tagline, short introduction, featured books and buttons leading to Explore Books
Explore Books	explore.html	Search box, category buttons, result counter and the grid of all eight book cards
Book Details	book-details.html?id=n	Cover, title, author, category, description and price of book n, with Add to Cart and Preview buttons
Book Preview	preview.html?id=n	Sample pages of book n in a reading view, with previous and next controls
Shopping Cart	cart.html	Selected books with quantity controls, remove buttons, line totals and grand total
Contact	contact.html	Store contact information and the enquiry form with validation messages

Table 4.2 — Pages and their purpose

4.3 Visual Design System

The visual identity is defined once as CSS custom properties on :root in the shared stylesheet and reused by every page. Bootstrap supplies the grid and component structure, while these tokens give the store its own colours, rounded corners and shadows. Using tokens means that a change of colour or spacing is made in one place and appears on all six pages.

Token	Use
--pp-primary	Main brand colour: navigation bar, headings and primary buttons
--pp-accent	Highlight colour: price text, badges, active category button and links on hover
--pp-bg	Soft paper-like page background
--pp-surface	Background of cards, the cart table and form panels
--pp-text / --pp-muted	Main body text and secondary text such as author names and hints
--pp-radius	Rounded corners of cards, buttons and inputs
--pp-shadow	Soft shadow that lifts book cards on hover

Table 4.3 — Main design tokens (style.css)

4.4 Book Data Model

Every book is a plain JavaScript object in the books array in js/books.js. Because all pages read from this array, adding or editing a book requires a change in one place only.

Field	Type	Purpose
id	number	Unique identifier (1 to 8), used in links such as book-details.html?id=3 and as the cart key
title, author	string	Display title and author name
category	string	Category used by the filter buttons and the search
price	number	Price of one copy; multiplied by quantity in the cart
description	string	Short description shown on the details page and in card summaries
cover	string	Path of the SVG cover file in assets/covers/
featured	boolean	Marks books shown in the featured section of the Home page
preview	array of strings	Sample pages of text shown on the Book Preview page

Table 4.4 — Fields of a book object

The shipped dataset holds eight books, each with its own original SVG cover. Because the filter buttons are built from the categories present in the data, adding a book in a new category creates a new button automatically.

4.5 Application Flow

The overall structure is a set of static pages that share four scripts and one stylesheet, as shown in Fig 4.1. Each page follows the same sequence when it loads:

- The page loads Bootstrap and style.css in the head and the scripts at the end of the body
- layout.js inserts the shared navigation bar and footer into the placeholder elements and marks the link of the current page as active
- cart.js reads the cart from localStorage and updates the item count shown in the navigation bar

- pages.js runs only the logic needed by the current page: featured books on Home, search and filtering on Explore, id lookup on Details and Preview, cart rendering on Cart, validation on Contact
- Whenever the cart changes, it is saved to localStorage and the count in the navigation bar is refreshed on the same page; other pages read the saved cart when they load

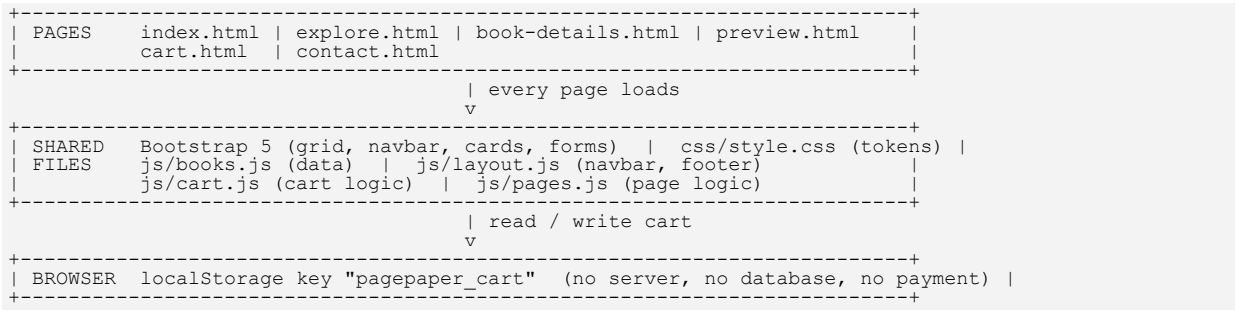


Fig 4.1 — Text diagram of the front-end architecture

4.6 Cart Data and Quantity Rules

The cart is stored as a JSON array of small objects containing only the book id and the quantity, for example [{"id":3,"qty":2}]. Titles, prices and covers are not duplicated; they are looked up from the books array when the cart is displayed, so the cart always shows current catalogue data. The rules applied by cart.js are summarised in Table 4.5.

Rule	Behaviour
Minimum quantity	A book in the cart always has a quantity of at least 1; to take a book out, the visitor uses Remove
Maximum quantity	A single book cannot exceed 10 copies; Add to Cart on a book already at 10 shows a notice and changes nothing
Typed values	A quantity typed into the field is converted to a whole number and corrected into the range 1 to 10; non-numeric input becomes 1
Line total	price multiplied by quantity for each book
Grand total	Sum of all line totals, shown with the total number of copies
Persistence	Saved to localStorage after every change and restored on the next page load
Corrupt data	If the stored text cannot be parsed, the cart is treated as empty instead of causing an error

Table 4.5 — Cart rules

5. Technology Stack

- **Markup:** HTML5 with semantic landmarks (header, nav, main, section, article, form, footer) on six pages
- **Styling:** CSS3 with custom properties, Flexbox, Grid, transitions and media queries, held in one shared stylesheet
- **Framework:** Bootstrap 5 — mobile-first grid, navbar, cards, buttons, badges, form controls and utility classes
- **Scripting:** Vanilla JavaScript (ES6+) — template literals, arrow functions, const/let, Array methods (find, filter, map, reduce), URLSearchParams, JSON and localStorage
- **Graphics:** Eight original SVG book covers created for the project

- **Data storage:** In-memory JavaScript array for the catalogue; localStorage for the cart; no database
- **Delivery:** Static files that run from the file system or any static web host
- **Development environment:** Any code editor and a modern browser with developer tools

6. Implementation

6.1 Page Structure and Shared Stylesheet

Each HTML page has the same skeleton: the Bootstrap stylesheet and style.css in the head, an empty header placeholder, a main element that holds the unique content of the page, an empty footer placeholder, and the scripts at the end of the body. style.css begins with the design tokens (Section 4.3) and then styles the book cards, cover images, price text, cart table and forms. The catalogue uses Bootstrap's grid so that the number of cards per row changes with the screen width.

```
<body>
  <div id="site-header"></div>

  <main class="container py-4">
    <div id="book-grid" class="row row-cols-1 row-cols-sm-2 row-cols-lg-4 g-4"></div>
  </main>

  <div id="site-footer"></div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
          bootstrap.bundle.min.js"></script>
  <script src="js/books.js"></script>
  <script src="js/layout.js"></script>
  <script src="js/cart.js"></script>
  <script src="js/pages.js"></script>
</body>
```

With row-cols-1, row-cols-sm-2 and row-cols-lg-4, the cards appear one per row on phones, two per row on small tablets and four per row on large screens, and the gap between them is set with the g-4 utility.

6.2 The Book Data Model

The first entry of the books array is shown below as a representative example of the structure (the text is shortened here only for layout).

```
const books = [
  {
    id: 1,
    title: "Sample Title",
    author: "Author Name",
    category: "Fiction",
    price: 299,
    featured: true,
    cover: "assets/covers/cover-1.svg",
    description: "A short description of the book shown on the details page...",
    preview: [
      "First sample page of the book...",
      "Second sample page of the book...",
      "Third sample page of the book..."
    ]
  },
  // ... seven more books
];
```

6.3 Shared Navigation and Footer

layout.js defines two functions that return the markup of the navigation bar and the footer. When the page has loaded, renderLayout() inserts them into the placeholders and adds the active class to the link whose address matches the current page. The navigation bar is a Bootstrap navbar with navbar-expand-lg, so it shows all links on large screens and collapses into a menu button on smaller ones. A badge next to the Cart link displays the total number of copies in the cart.

```
function renderLayout() {
  const page = location.pathname.split('/').pop() || 'index.html';
  document.getElementById('site-header').innerHTML = `
```

```
    <nav class="navbar navbar-expand-lg pp-navbar">
      <div class="container">
        <a class="navbar-brand" href="index.html">Page &amp;amp; Paper</a>
        <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#menu"
                aria-label="Toggle navigation">
          <span class="navbar-toggler-icon"></span></button>
        <div id="menu" class="collapse navbar-collapse">
          <ul class="navbar-nav ms-auto">
            ${navLink('index.html', 'Home', page)}
            ${navLink('explore.html', 'Explore Books', page)}
            ${navLink('cart.html', 'Cart <span id="cart-count" class="badge">0</span>', page)}
            ${navLink('contact.html', 'Contact', page)}
          </ul>
        </div>
      </div>
    </nav>`;
  document.getElementById('site-footer').innerHTML = footerMarkup();
  updateCartBadge();
}
```

6.4 Home Page

The Home page introduces the store with a hero section containing the store name, a short tagline and a button that leads to Explore Books. Below it, a featured section is created by filtering the books array for entries whose featured field is true and drawing each one as a card with its cover, title, author, price and a View Details button. Because the cards come from the same array as the catalogue, the featured section never needs separate content. A closing strip links to the catalogue and to the Contact page, so the page also acts as the main starting point for navigation.

6.5 Explore Books Page

The Explore Books page shows the catalogue. A row of category buttons (with an All button first) is generated from the distinct categories in the data, and a search box sits beside it. The page keeps two variables, the active category and the search text, and a single function applyFilters() applies both to the books array so that any combination works. The search text is compared in lower case with the title, author and category. The function then updates the "Showing n of 8 books" readout, shows a friendly empty-state message when nothing matches, and redraws the cards. Each card links to the details page and has an Add to Cart button.

```
function applyFilters() {
  const q = searchText.trim().toLowerCase();
  const result = books.filter(book => {
    const inCategory = activeCategory === 'All' || book.category === activeCategory;
    const inSearch = q === '' ||
      book.title.toLowerCase().includes(q) ||
      book.author.toLowerCase().includes(q) ||
      book.category.toLowerCase().includes(q);
    return inCategory && inSearch;
  });
  counter.innerHTML = `Showing <strong>${result.length}</strong> of ${books.length} books`;
  emptyState.classList.toggle('d-none', result.length > 0);
  grid.innerHTML = result.map(bookCard).join('');
}
```

6.6 Book Details Page

book-details.html is a single template that can display any book. When it loads, the script reads the id from the query string with URLSearchParams, converts it to a number and looks the book up in the array. If a book is found, the page fills in the cover, title, author, category badge, description and price and shows two buttons: Add to Cart, which calls the cart function, and Read Preview, which links to preview.html?id=n. If the id is missing or does not match any book, the page shows a clear "book not found" message with a link back to the catalogue instead of a blank page.

```
const params = new URLSearchParams(window.location.search);
const bookId = parseInt(params.get('id'), 10);
const book = books.find(b => b.id === bookId);

if (!book) {
  detailBox.innerHTML = '<p class="alert alert-warning">Book not found. ' +
                        '<a href="explore.html">Back to catalogue</a></p>';
} else {
  document.title = book.title + ' | Page & Paper';
  detailBox.innerHTML = bookDetailMarkup(book);
}
```

6.7 Book Preview Page

preview.html uses the same id lookup to find the book and then shows the strings of its preview array one page at a time inside a reading panel styled like a page of a book, with generous line spacing and a comfortable line length. Previous and Next buttons move between the sample pages and are disabled at the first and last page. A small label shows the current position (for example “Page 2 of 3”) and a progress bar fills as the visitor reads. At the end of the sample, a message explains that this is a preview only and offers Add to Cart and Back to Details buttons. The preview contains sample text only; no real e-book files are stored or delivered by this project.

6.8 Shopping Cart Page

cart.js holds all cart logic so that every page can use it. The cart is read from localStorage, changed in memory and written back. addToCart() adds one copy or increases the quantity, refusing to go beyond 10. changeQuantity() applies clampQty() to any value so that a quantity can never fall outside the range 1 to 10, and removeFromCart() deletes an entry. cart.html draws one row for each entry with the cover, title, unit price, a quantity control (minus button, number field and plus button), the line total and a Remove button. After every change the page is redrawn and the grand total and the item count are recalculated with reduce(). When the cart is empty the page shows a message and a link to Explore Books.

```
const CART_KEY = 'pagepaper_cart';
const MIN_QTY = 1;
const MAX_QTY = 10;

function loadCart() {
  try {
    const data = JSON.parse(localStorage.getItem(CART_KEY));
    return Array.isArray(data) ? data : [];
  } catch (err) {
    return [];
  }
}

function saveCart(cart) {
  localStorage.setItem(CART_KEY, JSON.stringify(cart));
  updateCartBadge();
}

function clampQty(value) {
  const n = parseInt(value, 10);
  if (isNaN(n)) return MIN_QTY;
  return Math.min(MAX_QTY, Math.max(MIN_QTY, n));
}

function addToCart(bookId) {
  const cart = loadCart();
  const item = cart.find(i => i.id === bookId);
  if (item) {
    if (item.qty >= MAX_QTY) { showNotice('You can buy at most 10 copies of a book.'); return; }
    item.qty += 1;
  } else {
    cart.push({ id: bookId, qty: 1 });
  }
  saveCart(cart);
}

function cartTotal() {
  return loadCart().reduce((sum, i) => sum + books.find(b => b.id === i.id).price * i.qty, 0);
}
```

6.9 Contact Page

The Contact page has two columns on large screens and stacks them on small ones. One column lists the store’s contact information as sample details for the demonstration; the other holds the enquiry form with name, e-mail, subject and message fields. On submit, the default action is cancelled and each field is checked; every problem is written in a small error line under its field. A valid submission clears the form and shows a confirmation message that states that the enquiry was recorded in the demonstration only. Nothing is sent to a server.

```
contactForm.addEventListener('submit', (e) => {
  e.preventDefault();
  let valid = true;
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
  if (!nameInput.value.trim()) {
    nameError.textContent = 'Please enter your name.'; valid = false;
  } else { nameError.textContent = ''; }

  if (!emailPattern.test(emailInput.value.trim())) {
    emailError.textContent = 'Please enter a valid e-mail address.'; valid = false;
  } else { emailError.textContent = ''; }

  if (messageInput.value.trim().length < 10) {
    messageError.textContent = 'Please write at least 10 characters.'; valid = false;
  } else { messageError.textContent = ''; }

  if (valid) { contactForm.reset(); successBox.classList.remove('d-none'); }
});
```

6.10 Original SVG Book Covers

The eight covers are original SVG illustrations stored as separate files in assets/covers/. Each uses a 3:4 view box, a gradient background, simple geometric shapes that suggest the theme of the book, and the title set as SVG text. Because they are vector graphics, they stay sharp on every screen, add very little to the page weight, and avoid any dependence on copyrighted images. The cards, the details page and the cart all display the same cover file through an img element whose alt text names the book, and CSS keeps the 3:4 shape with object-fit and aspect-ratio.

6.11 Responsive Layout and Accessibility

- Bootstrap's mobile-first grid places catalogue cards in one, two or four columns depending on the screen width; the details page shows the cover above the text on phones and beside it on larger screens
- The navbar collapses into a menu button below the large breakpoint, and the cart table adapts to narrow screens so that the controls remain reachable
- Semantic landmarks, one h1 per page, labelled form controls (label elements linked to inputs) and alt text for every cover support keyboard and screen-reader users
- Buttons and links are real button and a elements, and Bootstrap's visible focus styles are kept
- Quantity controls have aria-labels such as "Increase quantity", and the cart count badge and notices are updated as text so that changes can be announced by assistive technology

6.12 Functional Modules

1. Layout and Navigation Module

- Shared navbar and footer, active-link highlighting, cart badge and the shared design-token stylesheet

2. Catalogue Module

- books array, card renderer, category buttons, live search, result counter and empty state

3. Book Details and Preview Module

- Query-string lookup, details template, not-found handling, paged sample reading view and progress bar

4. Shopping Cart Module

- localStorage persistence, add, change quantity (1 to 10), remove, line totals and grand total

5. Contact Module

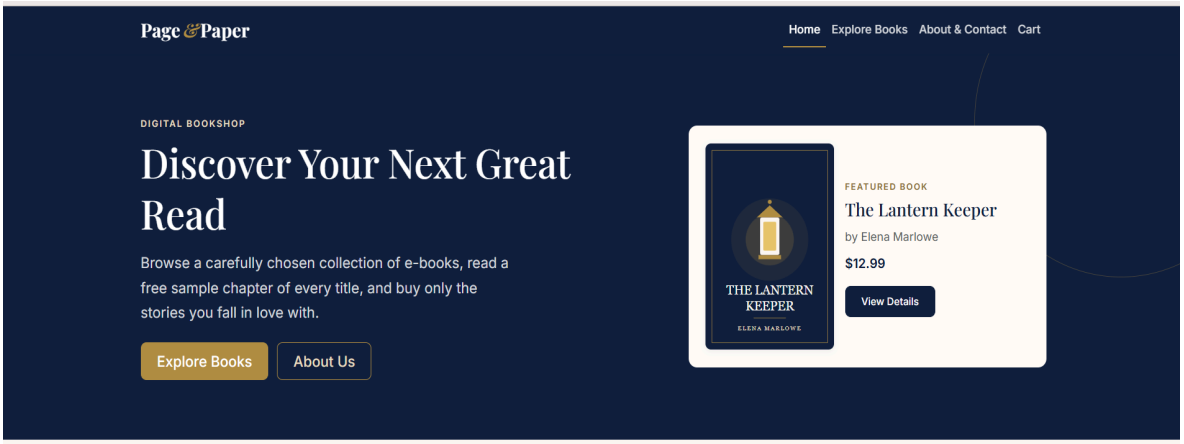
- Contact information panel, field validation, error messages and demonstration confirmation

7. Results and Discussion

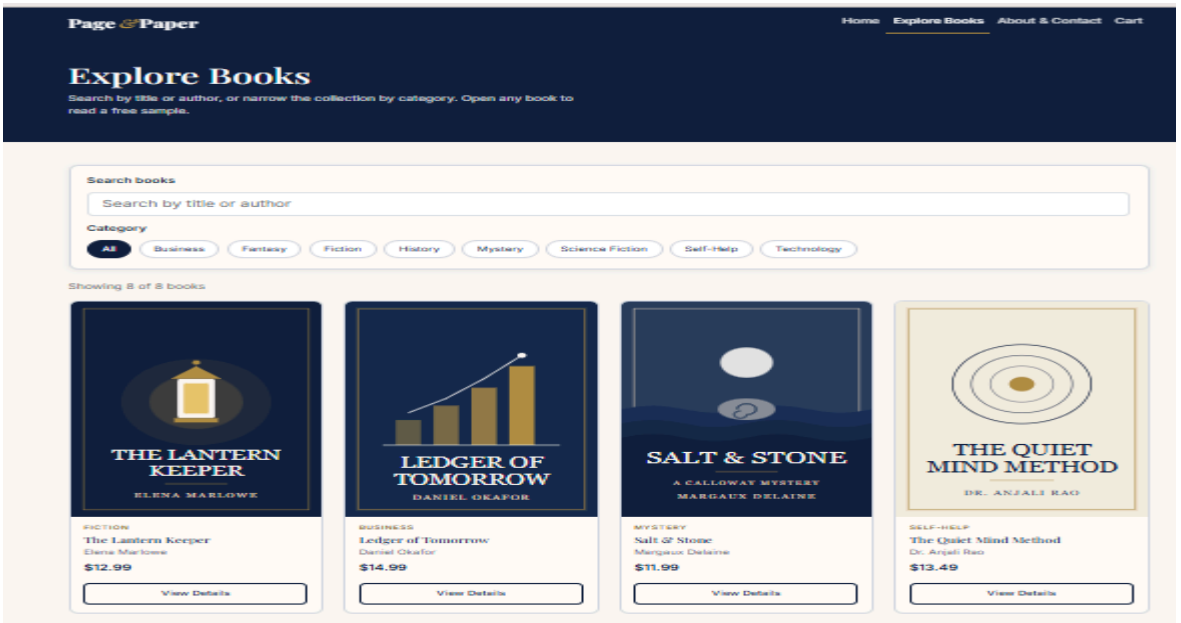
7.1 Overview of the Delivered Site

The final deliverable is a complete, multi-page e-book store front end made of six HTML pages, one shared stylesheet, four JavaScript files and eight SVG covers. Instead of screenshots, this chapter describes each page as a visitor sees it, region by region, and then summarises the results in Table 7.3.

7.2 Home Page

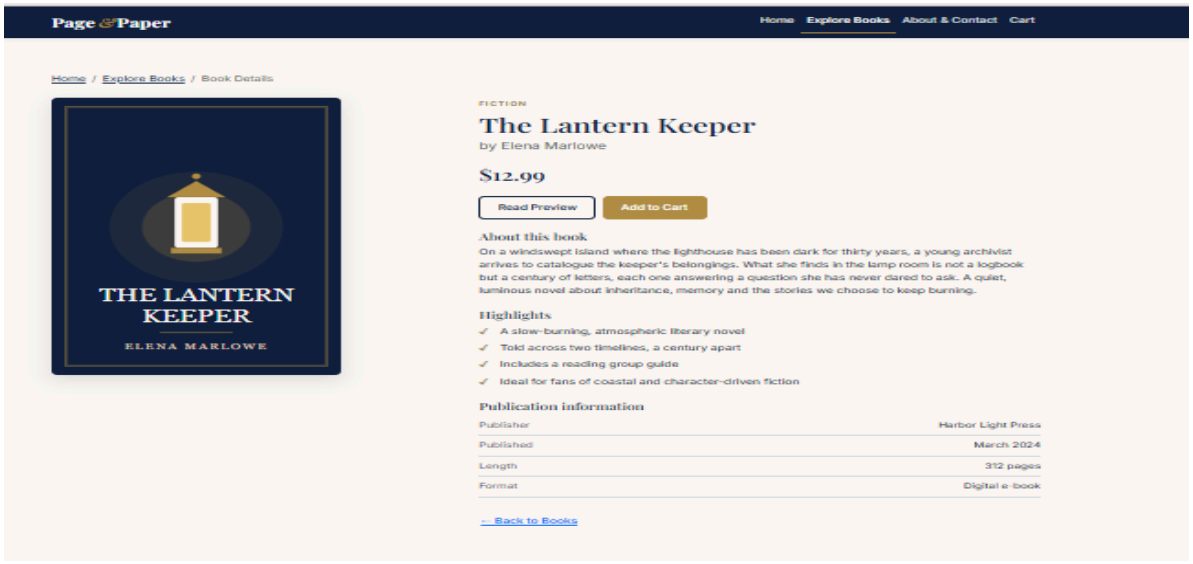


7.3 Explore Books Page



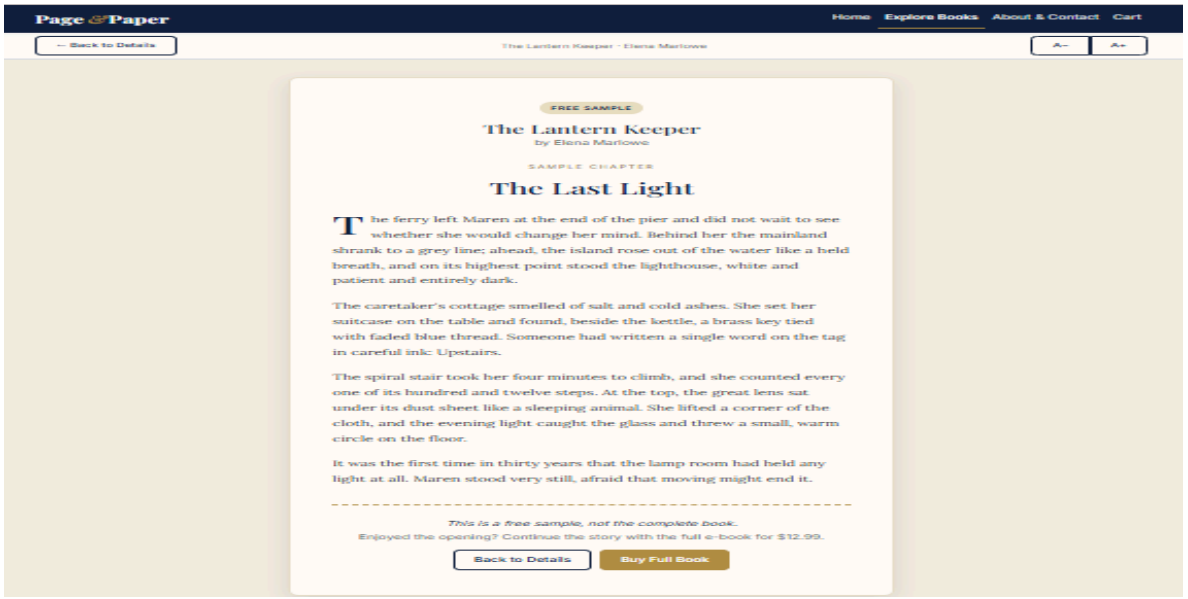
The page opens with a heading, the search box and the category buttons, with All selected. The readout above the grid shows the number of books displayed, and the grid shows all eight cards. Typing in the search box narrows the grid after each keystroke, selecting a category button limits the cards to that category, and the two can be combined. When nothing matches, the grid is replaced by an empty-state message and the readout shows zero books. Choosing All and clearing the search box restores the full catalogue.

7.4 Book Details Page



The page shows the large SVG cover on one side and, on the other, the title, author, category badge, description and price, followed by the Add to Cart and Read Preview buttons. Opening the page without a valid id shows the not-found message instead of an empty page. Because one template serves all eight books, a change of layout is made once and applies to every book.

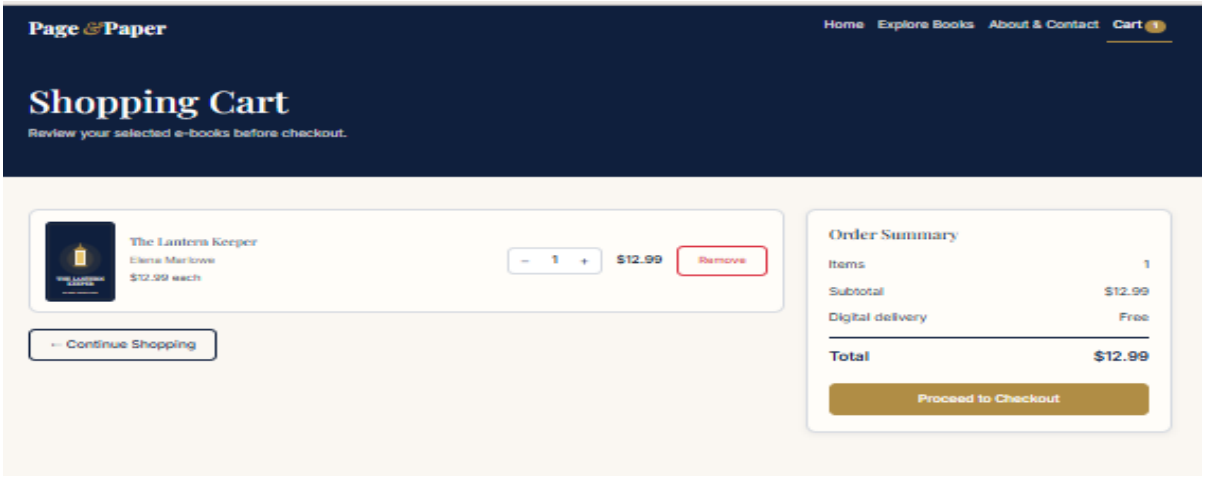
7.5 Book Preview Page



The page presents the book title and a reading panel containing one sample page of text. Previous and Next buttons move through the sample pages, the position label and the progress bar update, and the buttons are disabled at the ends. After the last sample page, the visitor sees a closing note and buttons to add the book to the cart or return to its details.

7.6 Shopping Cart Page

The cart page lists every selected book and, below the list, the totals. The layout is represented in the text diagram below.



When the quantity field is changed, the page corrects values outside 1 to 10, recalculates the line total and the grand total and saves the cart. Reloading the page or opening another page keeps the cart contents because they are read back from localStorage. The Remove button deletes a row, and when the last row is removed, an empty-cart message with a link to Explore Books appears.

7.7 Contact Page

The page shows a contact information panel (address, e-mail and phone entries given as sample details for the demonstration) next to the enquiry form with name, e-mail, subject and message fields. Submitting an empty form shows an error under each required field, an invalid e-mail address shows its own message, and a valid form is cleared and followed by a confirmation notice.

7.8 Responsive Behaviour

Screen width	Navigation	Catalogue grid	Other adjustments
Below 576 px (phone)	Collapsed menu button	1 card per row	Details page stacks cover above text; contact columns stack
576 to 991 px (tablet)	Collapsed menu button	2 cards per row	Cart controls remain on one line per item
992 px and above (desktop)	Full horizontal menu	4 cards per row	Details page shows cover and text side by side; contact in two columns

Table 7.2 — Responsive behaviour of the interface

7.9 Summary of Results

Aspect	Target	Outcome
Site structure	Multi-page store, not a single page	Six linked HTML pages with one shared stylesheet
Shared layout	Navbar and footer written once	Inserted on every page by layout.js; active link highlighted
Catalogue	Eight books with original covers	Eight book objects and eight original SVG covers
Search and filtering	Text search, category filter and both together	One applyFilters() function; counter and empty state
Details and preview	Per-book information and sample reading	One template each, driven by the id in the URL
Cart	Add, update, remove and totals	Implemented with localStorage persistence
Quantity limit	Between 1 and 10	Enforced by clampQty() on buttons, typed values and Add to Cart
Contact form	Validate and confirm	Name, e-mail and message validated; demonstration confirmation

Aspect	Target	Outcome
Responsiveness	Phone, tablet and desktop	Bootstrap grid and navbar collapse; layout adapts at standard breakpoints
Back-end features	Not in scope	No payment, login, database or order processing, by design
Offline use	Usable without internet	Pages open locally, but Bootstrap loads from a CDN and needs a connection unless saved locally

Table 7.3 — Summary of results

8. Challenges and Issues Faced

Building the site raised two kinds of challenge: technical problems that were solved during implementation, and limitations that remain because the project is a front-end demonstration.

8.1 Technical Problems Solved

Repeated navigation bar and footer

- Copying the same navbar and footer into six pages made every change error-prone. Resolution: the markup is written once in layout.js and inserted into placeholders when each page loads.

Passing the selected book between pages

- The details and preview pages must know which book was chosen, but plain HTML pages share no variables. Resolution: the book id is added to the link (book-details.html?id=3) and read with URLSearchParams; an unknown id shows a not-found message.

Combining filter and search

- The category buttons and the live search had to work together without stale state. Resolution: both are stored as variables and applied together inside one applyFilters() function that also updates the counter and the empty state.

Keeping the cart consistent across pages

- The cart count in the navbar and the cart page must always agree. Resolution: all pages use the same functions in cart.js, the cart is saved after every change, and the badge is refreshed from the stored data on each page load.

Enforcing the quantity limit

- A visitor can press buttons, type any text into the quantity field or press Add to Cart repeatedly. Resolution: a single clampQty() function converts every value to a whole number between 1 and 10, and addToCart() refuses to pass 10.

Handling damaged stored data

- If the saved cart text is missing or damaged, JSON.parse can throw an error. Resolution: loadCart() uses try/catch and returns an empty cart in that case.

Responsive cart and cards

- Long titles and the quantity controls made the cart crowded on narrow screens. Resolution: Bootstrap grid classes and flexible widths let rows wrap, and a fixed 3:4 cover size keeps cards even.

8.2 No Payment, Login or Order Processing

- The project has no payment gateway, no user accounts and no order handling, because these require a server. The cart ends at the totals. Recommendation: for a production store, connect the cart to a back end with a payment provider, authentication and an order database.

8.3 Cart Stored Only in the Browser

- localStorage belongs to one browser on one device, can be cleared by the user and is not secure. A cart started on a phone does not appear on a laptop. Recommendation: keep carts on a server for signed-in customers.

8.4 Contact Form Not Transmitted

- The form is validated and a confirmation is shown, but nothing is stored or e-mailed because there is no back end. Recommendation: connect the form to a serverless function or form-handling service.

8.5 Sample Content Only

- Book preview text, descriptions, prices and contact details are sample content created for the project; no real e-book files are stored or delivered. The catalogue lives in a JavaScript array, which is practical for eight books but would need a database for a large store.

8.6 Dependence on External Bootstrap Files

- Bootstrap is linked from a CDN, so a first visit without internet shows unstyled pages. Recommendation: download the Bootstrap files into the project folder and link them locally.

8.7 Accessibility Notes

- Semantic markup, labels, alt text and Bootstrap's focus styles were used, but a full audit with automated tools and screen readers has not been carried out. Cards are built with innerHTML from the developer-written array, which is safe now but would require escaping if the data came from users.

9. Testing

The site was designed with the test cases in Table 9.1 in mind. They cover navigation, the catalogue, the details and preview pages, the cart and its quantity limits, the contact form and responsive behaviour. Each case is executed manually in the browser by performing the stated action and comparing what happens with the expected outcome; the developer tools are used to inspect the stored cart.

No.	Test case	Action / input	Expected outcome
1	Load Home page	Open index.html	Hero, featured books and shared navbar and footer appear
2	Shared navigation	Click each navbar link in turn	Each of the six pages opens; the current link is highlighted
3	Explore catalogue	Open explore.html	Eight cards shown; readout says 8 of 8 books
4	Category filter	Click one category button	Only books of that category remain; readout updates
5	Search by title	Type part of a title	Matching book(s) shown immediately

No.	Test case	Action / input	Expected outcome
6	Search by author	Type part of an author name	Books by that author are shown
7	Search and category combined	Select a category, then type a keyword	Only books satisfying both conditions are shown
8	No match	Type a word that appears in no book	Empty-state message shown; readout shows 0 books
9	Reset filters	Select All and clear the search box	All eight books are shown again
10	Open book details	Click View Details on a card	Details page shows cover, title, author, category, description and price
11	Invalid book id	Open book-details.html?id=99	Not-found message with link to the catalogue
12	Open preview	Click Read Preview on the details page	Preview page opens on page 1 of the sample
13	Preview navigation	Click Next and Previous	Page label and progress bar change; buttons disabled at the ends
14	Add to cart	Click Add to Cart on a book	Navbar badge increases by 1; cart stored in localStorage
15	Add same book again	Click Add to Cart twice on one book	Quantity increases to 2, no duplicate row
16	Cart persistence	Add a book, then reload the page	Cart contents and badge are unchanged
17	Increase quantity	Click + in the cart	Quantity, line total and grand total increase
18	Maximum quantity	Raise a quantity to 10, then click + or Add to Cart	Quantity stays at 10 and a notice is shown
19	Minimum quantity	Click - when quantity is 1	Quantity stays at 1; row is not removed
20	Typed quantity	Type 0, 25 and abc into the quantity field	Values are corrected to 1, 10 and 1 respectively
21	Remove book	Click Remove on a row	Row disappears; totals and badge update
22	Empty cart	Remove all books	Empty-cart message with link to Explore Books
23	Empty contact form	Submit with all fields empty	Error message under each required field
24	Invalid e-mail	Enter an address without @ and submit	E-mail error message shown
25	Valid enquiry	Fill all fields correctly and submit	Confirmation shown and form cleared; nothing is sent
26	Phone layout	View at about 390 px width	One card per row, menu button, no horizontal scrolling
27	Desktop layout	View at 1366 px width	Four cards per row and horizontal menu

Table 9.1 — Test cases

The outcome of each case should be recorded against the final build before submission. Cross-browser testing on Firefox and Safari, automated accessibility audits and user-acceptance testing are listed under future work.

Appendix A — Markup Excerpts (HTML pages)

The excerpts below are representative of the markup used by the pages; element ids follow the scripts described in Chapter 6.

Category filter toolbar and search box (explore.html)

```
<section class="mb-4">
  <div class="row g-3 align-items-center">
    <div class="col-lg-8">
      <div id="category-buttons" class="d-flex flex-wrap gap-2" role="group"
           aria-label="Filter books by category">
        <!-- buttons are generated from the categories in books.js -->
```

```
      </div>
    </div>
    <div class="col-lg-4">
      <input type="search" id="search-input" class="form-control"
             placeholder="Search title, author or category" aria-label="Search books">
    </div>
  </div>
  <p id="result-counter" class="text-muted mt-3" aria-live="polite"></p>
</section>
<div id="empty-state" class="alert alert-info d-none">No books match your search.</div>
<div id="book-grid" class="row row-cols-1 row-cols-sm-2 row-cols-lg-4 g-4"></div>
```

Book card template (generated by JavaScript)

```
function bookCard(book) {
  return `
    <div class="col">
      <article class="card h-100 pp-card">
        <img src="${book.cover}" class="card-img-top pp-cover" alt="Cover of ${book.title}">
        <div class="card-body">
          <span class="badge pp-badge">${book.category}</span>
          <h2 class="h6 card-title mt-2">${book.title}</h2>
          <p class="text-muted small mb-1">${book.author}</p>
          <p class="pp-price">${book.price}</p>
        </div>
        <div class="card-footer d-flex gap-2">
          <a href="book-details.html?id=${book.id}"
             class="btn btn-outline-primary btn-sm">Details</a>
          <button class="btn btn-primary btn-sm"
                  onclick="addToCart(${book.id})">Add to Cart</button>
        </div>
      </article>
    </div>`;
}
```

Enquiry form (first rows of contact.html)

```
<form id="contact-form" novalidate>
  <div class="mb-3">
    <label for="name" class="form-label">Your Name</label>
    <input type="text" id="name" class="form-control" required>
    <div id="name-error" class="text-danger small"></div>
  </div>
  <div class="mb-3">
    <label for="email" class="form-label">E-mail Address</label>
    <input type="email" id="email" class="form-control" required>
    <div id="email-error" class="text-danger small"></div>
  </div>
  <div class="mb-3">
    <label for="subject" class="form-label">Subject</label>
    <input type="text" id="subject" class="form-control">
  </div>
</form>
```

Appendix B — Design Tokens and Responsive Rules (style.css)

Design tokens (excerpt)

```
:root {
  --pp-primary: #2b3a55;
  --pp-accent:  #e0a458;
  --pp-bg:      #faf7f2;
  --pp-surface: #ffffff;
  --pp-text:    #1f2430;
  --pp-muted:   #6b7280;
  --pp-radius:  12px;
  --pp-shadow:  0 6px 18px rgba(31, 36, 48, 0.12);
}
```

Card, cover and responsive rules (excerpt)

```
body { background: var(--pp-bg); color: var(--pp-text); }

.pp-card {
  border: 0;
  border-radius: var(--pp-radius);
  background: var(--pp-surface);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.pp-card:hover { transform: translateY(-4px); box-shadow: var(--pp-shadow); }

.pp-cover { aspect-ratio: 3 / 4; object-fit: cover; }
.pp-price { color: var(--pp-accent); font-weight: 700; }

@media (max-width: 767.98px) {
  .cart-row { flex-direction: column; align-items: flex-start; }
  .cart-row .qty-control { margin-top: 0.5rem; }
}
```

Appendix C — Interaction Logic (JavaScript)

Quantity controls in the cart page

```
function changeQuantity(bookId, newValue) {
  const cart = loadCart();
  const item = cart.find(i => i.id === bookId);
```

```
  if (!item) return;
  item.qty = clampQty(newValue);
  saveCart(cart);
  renderCart();
}

function removeFromCart(bookId) {
  saveCart(loadCart().filter(i => i.id !== bookId));
  renderCart();
}

function updateCartBadge() {
  const badge = document.getElementById('cart-count');
  if (badge) badge.textContent = loadCart().reduce((n, i) => n + i.qty, 0);
}
```

Preview page navigation

```
let pageIndex = 0;

function showPage() {
  previewText.textContent = book.preview[pageIndex];
  pageLabel.textContent = `Page ${pageIndex + 1} of ${book.preview.length}`;
  progressBar.style.width = ((pageIndex + 1) / book.preview.length) * 100 + '%';
  prevBtn.disabled = pageIndex === 0;
  nextBtn.disabled = pageIndex === book.preview.length - 1;
}

prevBtn.addEventListener('click', () => { pageIndex--; showPage(); });
nextBtn.addEventListener('click', () => { pageIndex++; showPage(); });
```

10. Conclusion and Future Scope

10.1 Conclusion

PAGE & PAPER demonstrates that a complete, attractive and interactive multi-page e-book store interface can be built with HTML5, CSS3, Bootstrap 5 and JavaScript alone. The shared navigation and footer, the data-driven catalogue with combined search and filtering, the details and preview pages driven by the URL, the localStorage cart with its quantity limit and the validated contact form work together as a coherent shopping experience. The use of design tokens, a single book array and small, well-named functions keeps the code easy to read and modify.

The project also gave practical experience with semantic HTML, the Bootstrap grid and components, responsive design, the DOM, query-string parameters, Web Storage, SVG graphics and form validation, and showed the value of defining clear rules, such as the 1 to 10 quantity range, and testing them from every direction.

10.2 Known Limitations

- There is no payment gateway, authentication, database or real order processing; the project is a front-end demonstration
- The cart is stored only in the visitor’s own browser and does not follow the visitor to another device
- Contact enquiries are validated and confirmed on screen but are not transmitted, stored or e-mailed
- Preview text, descriptions, prices and contact details are sample content; no real e-book files are provided
- Bootstrap is loaded from a CDN, so a first visit needs an internet connection
- Testing was carried out manually; cross-browser, automated and screen-reader testing remain to be done

10.3 Future Enhancements

- Add a back end with a database for books, customers and orders, and replace the JavaScript array with an API

- Add user registration and login, and keep carts and purchase history on the server
- Integrate a payment provider and generate real order confirmations and secure e-book downloads
- Connect the contact form to e-mail or a ticketing service
- Add sorting by price, wish lists, ratings and reviews, and pagination for a larger catalogue
- Serve Bootstrap locally, add a service worker for offline use and run automated accessibility and performance audits

11. References

- MDN Web Docs — HTML, CSS and JavaScript reference, Mozilla. https://developer.mozilla.org
- Bootstrap Team — Bootstrap 5.3 Documentation (grid, navbar, cards, forms and utilities). https://getbootstrap.com/docs/5.3/
- WHATWG — HTML Living Standard (Web Storage and URL Standard). https://html.spec.whatwg.org
- MDN Web Docs — Window.localStorage and URLSearchParams. https://developer.mozilla.org
- W3C — Scalable Vector Graphics (SVG) 2. https://www.w3.org/TR/SVG2/
- W3C — Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
- W3C — CSS Grid Layout and CSS Flexible Box Layout Modules. https://www.w3.org/TR/css-grid-1/ and https://www.w3.org/TR/css-flexbox-1/
- Marcotte, E. — “Responsive Web Design”, A List Apart, May 2010.

12. System Usage / Use Case Description

The system has one actor, the Visitor, who browses the store, reads about and previews books, builds a cart and sends an enquiry. The store owner who would receive enquiries and orders in a real system is outside the scope of this front-end demonstration. Every interaction runs in the browser.

Use case	Description
Browse featured books	Visitor opens Home and sees the introduction, the featured book cards and the links to other pages.
Search and filter	Visitor opens Explore Books, selects a category and/or types a keyword; the grid and result counter update immediately and an empty state appears when nothing matches.
View book details	Visitor opens a book; the details page shows cover, title, author, category, description and price.
Read a preview	Visitor selects Read Preview and moves through the sample pages with the Previous and Next buttons.
Add to cart	Visitor selects Add to Cart on a card or on the details page; the navbar badge is updated and the cart is saved in the browser.
Manage the cart	Visitor changes a quantity between 1 and 10 or removes a book; line totals and the grand total are recalculated.
Send an enquiry	Visitor completes name, e-mail and message; the system validates the fields and shows a demonstration confirmation.

Table 12.1 — Use cases

12.1 Comparison with Traditional Methods

Criterion	Visiting a physical shop / single-page listing	PAGE & PAPER website
Finding a book	Walk the shelves or scroll one long page	Category buttons and live keyword search
Book information	Cover blurb or short caption	Dedicated details page with author, category, description and price
Trying before buying	Browse a few pages in the shop	Sample reading view with page navigation
Choosing several books	Carry books to the counter	Cart with quantities from 1 to 10, line totals and grand total
Access	Shop opening hours and location	Any device with a browser, at any time
Cost and infrastructure	Premises, stock and staff	Static files; no server required for the demonstration

Table 12.2 — Comparison with traditional methods

13. User Manual — Running the Application

Step 1: Prepare the files

Place the six HTML pages and the css, js and assets folders together in one project folder. Any current version of Chrome, Edge or Firefox may be used. An internet connection is needed on first use to load Bootstrap from the CDN.

Step 2: Project structure

```
pagepaper/
  index.html            Home page
  explore.html          Explore Books page
  book-details.html     Book Details page
  preview.html          Book Preview page
  cart.html             Shopping Cart page
  contact.html          Contact page
  css/style.css         shared stylesheet
  js/books.js           books array
  js/layout.js          shared navbar and footer
  js/cart.js            cart functions
  js/pages.js           page-specific logic
  assets/covers/        eight SVG book covers
```

Step 3: Open the site

Double-click index.html. For the most reliable localStorage behaviour, optionally serve the folder locally:

```
cd pagepaper
python -m http.server 8000
# then open http://localhost:8000
```

Step 4: Use the site

- Use the navigation bar to move between pages; on a phone, tap the menu button first
- On Explore Books, choose a category or type in the search box, then select View Details on a book
- On the details page, read the information, select Read Preview to sample the book, or select Add to Cart
- In the preview, use Previous and Next to move through the sample pages
- Open Cart to change quantities (1 to 10), remove books and check the totals
- Open Contact, fill in name, e-mail and message and submit the enquiry form to see the validation and confirmation

Step 5: Customise

- Edit the books array in js/books.js to change titles, authors, categories, prices, descriptions and preview text

- Change colours and spacing through the custom properties at the top of css/style.css
- Replace the SVG files in assets/covers/ with new cover illustrations and update the cover paths in the data

14. Glossary of Terms

Term	Definition
HTML5	Current version of the HyperText Markup Language, used to structure the pages with semantic elements
CSS3 / Custom Properties	Style language; custom properties (variables such as --pp-primary) store design tokens for reuse
Bootstrap 5	Front-end framework providing a responsive grid, components and utility classes
Responsive design	Approach in which one page adapts its layout to different screen sizes
Breakpoint	Screen width at which the layout changes, for example the medium and large breakpoints of Bootstrap
DOM	Document Object Model — the tree of page elements that scripts can read and change
Query string	Part of a URL after the question mark, such as ?id=3, used here to tell a page which book to show
localStorage	Browser storage that keeps small amounts of text data between visits for one site
JSON	JavaScript Object Notation — text format used to save the cart in localStorage
SVG	Scalable Vector Graphics — XML-based image format used for the book covers
CDN	Content Delivery Network — servers from which shared libraries such as Bootstrap are loaded
Cart	List of the books a visitor has chosen, with quantities and totals
ARIA	Accessible Rich Internet Applications attributes that expose roles and states to assistive technology
WCAG	Web Content Accessibility Guidelines published by the W3C

SDG Mapping

Project Title: PAGE & PAPER – E-BOOK SALES AND PREVIEW WEBSITE: A RESPONSIVE MULTI-PAGE E-BOOK STORE USING HTML, CSS, BOOTSTRAP AND JAVASCRIPT

SDG (No. & Theme)	Activity addressing SDG Theme
SDG 4 – Quality Education	Makes books easy to discover and lets readers sample content before choosing, supporting access to reading and learning materials.
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates lightweight digital adoption: a static, framework-supported front end that needs no server and is maintained by editing a single data array.
SDG 10 – Reduced Inequalities	Responsive layout, semantic markup, labelled controls and alternative text allow visitors with different devices and abilities to use the store.
SDG 12 – Responsible Consumption and Production	Presents books as digital e-books, an approach that avoids printing, packaging and transport of paper copies.